Práctica 2. Sistemas de Percepción. Detección de objetos usando el pipeline de reconocimiento tradicional

Aitor Ruiz Bautista Juan Pérez Fernández

13 de octubre de 2026

1. Introducción

La detección y alineación de objetos en entornos tridimensionales es una tarea central en los sistemas de percepción para aplicaciones en robótica, reconstrucción 3D y navegación autónoma. En esta práctica, se implementa un pipeline de reconocimiento tradicional sobre nubes de puntos, utilizando la biblioteca *Open3D* y el lenguaje Python, con el objetivo de localizar objetos previamente segmentados dentro de una escena compleja.

El procedimiento seguido se basa en una secuencia de etapas clásicas ampliamente estudiadas: eliminación de planos dominantes, reducción de densidad mediante *downsampling*, detección de puntos clave con ISS, extracción de descriptores FPFH, emparejamiento robusto de correspondencias mediante RANSAC y refinamiento final con ICP. Todo ello permite estimar la transformación rígida óptima que alinea cada objeto con su posición en la escena.

Este proyecto pone en práctica conceptos fundamentales de percepción artificial, a la vez que permite analizar el impacto de distintos parámetros sobre la calidad del registro, incluyendo la sensibilidad al ruido, la cantidad de puntos clave detectados o la estabilidad del alineamiento. Además, se presenta un anexo con la aproximación empleada para la segmentación 3D de un objeto dentro de una escena.

2. Estado del Arte

La alineación de nubes de puntos es una técnica fundamental para la reconstrucción y representación precisa de entornos en espacios tridimensionales virtuales. Al capturar datos mediante sensores 3D, como cámaras RGB-D, LIDAR o escáneres láser, se obtienen múltiples vistas parciales de un mismo escenario. Para generar una representación unificada y coherente del entorno, es necesario alinear dichas nubes de puntos en un sistema de coordenadas común.

Existen enfoques capaces de reconstruir escenas 3D a partir de imágenes 2D, como Structure from Motion (SfM), sin embargo, en este trabajo se abordará específicamente el uso de datos tridimensionales adquiridos directamente por sensores 3D, lo que permite una mayor precisión y densidad en la representación geométrica.

Este proceso de alineación se apoya en algoritmos ampliamente estudiados e implementados en bibliotecas como Open3D, que permiten automatizar el registro de nubes de puntos mediante un pipeline tradicional basado en la detección de puntos clave, cálculo de descriptores, emparejamiento robusto y refinamiento iterativo.

En la actualidad, la alineación de nubes de puntos 3D continúa siendo un área de investigación activa, especialmente impulsada por los avances en técnicas de aprendizaje profundo. Si bien esta tecnología ya cuenta con aplicaciones consolidadas en campos como la robótica, los videojuegos [1], la navegación autónoma y la reconstrucción arquitectónica, el uso de redes neuronales ha abierto nuevas posibilidades en términos de precisión, robustez y generalización. Modelos como DCP o 3DMatch han superado las limitaciones de los enfoques clásicos al aprender representaciones discriminativas directamente de datos reales, permitiendo mejorar el emparejamiento de puntos incluso en presencia de ruido, oclusiones o cambios de escala. Este tipo de soluciones resulta especialmente relevante en contextos dinámicos, como la robótica móvil o la manipulación autónoma, donde se requiere registrar escenas complejas en tiempo real. A nivel industrial, empresas como Ubisoft también han incorporado tecnologías de escaneo y alineación 3D para digitalizar entornos físicos y recrearlos en experiencias virtuales inmersivas.

2.1. Fundamentación Teórica

El proceso de alineación de nubes de puntos 3D mediante el pipeline tradicional se basa en diferentes algoritmos implementados en métodos de la librería Open3D:

- **Intrinsic Shape Signatures (ISS)**[2]: Se emplea en la obtención de puntos claves robustos, invariantes a rotación y resistentes al ruido. Este método evalua la dispersión geométrica local de un punto mediante la matriz de covarianza en una vecindad especificada, seleccionando puntos con alta curvatura intrínseca a través de la relación entre los autovalores.
- **Fast Point Features Histograms (FPFH)** [3]: Codifican la geometría local de cada punto de interés mediante histogramas de relaciones angulares entre vecinos para generar los descriptores. Estos permiten comparar regiones similares entre el modelo y la escena incluso en presencia de ruido u oclusiones. Su cálculo se realiza mediante el ajuste del radio de vecindad y la estructura KD-Tree.
- **RANdom SAmple Consensus (RANSAC)** [4] : Se emplea para calcular una transformación inicial basada en las correspondencias entre descriptores. A través del ajuste iterativo sobre un subconjunto aleatorio se

ajusta el modelo candidato cuya transformación maximice el número de inliers (correspondencias buenas). De cara a evitar un sobreajuste se emplea un criterio de convergencia basado en la precisión deseada del modelo. El modelo resultante proporciona un subóptimo de la matriz de transformación entre la nube de puntos del objeto y la escena.

- **Iterative Closest Points (ICP)** [5]: Se emplea para refinar la transformación robusta obtenida mediante RANSAC. Este algoritmo iterativo minimiza la distancia entre las correspondencias en una vecindad mediante SVD (Singular Value Decomposition), optimizando así la alineación de las nubes. A partir del subóptimo de RANSAC, ICP garantiza convergencia a un mínimo local.

2.2. Aplicaciones y Proyectos Relevantes

La alineación de nubes de puntos es clave en diversas tareas del ámbito de la robótica como reconstrucción 3D, localización, navegación y manipulación. En el estado del arte actual, los enfoques van más ligados a las técnicas de Deep Learning, dejando en desuso el pipeline tradicional. Destacar las siguientes aplicaciones y sus proyectos correspondientes:

- **Deep Closest Point (DCP)** [6] es un enfoque basado en redes neuronales que emplea una arquitectura basada en PointNet y atención tipo Transformer para estimar transformaciones rígidas de forma robusta sin depender de descriptores tradicionales. Una posible aplicación de este enfoque es el proceso de mapeado en robótica móvil mediante sensores LiDAR.

- **3DMatch** aprende descriptores 3D directamente de datos escaneados, permitiendo reconocer y alinear fragmentos de nubes de puntos en condiciones reales. Se ha demostrado su eficacia en reconstrucciones a partir de cámaras RGB-D [7] lo cual puede ser útil en tareas de SLAM visual para robótica móvil y navegación mediante UAVs.

- **LOAM** [8] es un sistema de localización y mapeo en tiempo real para sensores LiDAR, que realiza alinecación continua entre escaneos usando características geométricas. Este enfoque puede tener mucha relevancia en la navegación autónoma de vehículos terrestres y drones en entornos no estructurados.

- **Pose Registration using INliers (PRIN)** [9] combina la eficiencia de redes neuronales con las restricciones geométricas para alinear objetos parcialmente visibles o deformables. Este enfoque podría resultar interesante en tareas de manipulación o ensamblaje en escenarios industriales.

3. Metodología

El objetivo es encontrar la transformación entre dos nubes de puntos para que queden alineadas. Para ello se debe seguir el siguiente procedimiento:

3.1. Reducción de la nube de puntos

En primer lugar se debe hacer un proceso de *downsampling*, que consiste en reducir la densidad de puntos de la nube. Esto se hace por varias razones:

- **Reducción del coste computacional:** Una nube de puntos muy densa incrementa el tiempo de procesamiento y el consumo de memoria.
- **Eliminación de redundancia :** Puntos muy cercanos entre si pueden ser redundantes y añadir ruido.
- **Estabilidad :** Al reducir la cantidad de puntos se mejora la estabilidad evitando sobreajustes locales y favoreciendo una convergencia más suave.

3.2. Eliminación de planos dominantes

Para poder centrarse en los objetos los cuales queremos detectar, se hace una eliminación de los planos para facilitar y agilizar todo el proceso. Como consecuencia, obtenemos una escena con menor cantidad de puntos conservando los objetos a detectar. Una parte clave de este proceso es regular el umbral de distancia que regula si un punto pertenece a un plano dado que aquellos objetos colocados sobre una mesa, si pierden demasiados puntos de su nube se complica el proceso de emparejamiento al tener menos puntos de interés. [Incluir FOTOS antes y despues de eliminar planos]

3.3. Detección de puntos de interés

Para la detección de los puntos de interés se hace uso de ISS mediante el método proporcionado por open3D [10]. Primero, tanto para la nube de la escena como para la nube del objeto, se especifica un radio y un número de puntos vecinos para estimar las normales [11] mediante la estructura KD-Tree y búsqueda por KNN.

Una vez obtenidas, se especifican una serie de parámetros para obtener los puntos clave de escena y objeto. Estos parámetros, que actúan como umbrales para descartar puntos, son *salient radius*; que especifica el radio de vecindad del punto de interés, *non max radius*; que suprime puntos cercanos y similares entre sí, *gamma21* y *gamma32*; que son relaciones entre los autovalores del tensor para eliminar formas alargadas o planas. Para la escena se han fijado a lo largo de la experimentación los mismos valores, mientras que para obtener un buen descriptor de cada objeto se han especificado diferentes valores con el fin de maximizar el número de puntos claves obtenidos. Con el fin de verificar la obtención de suficientes puntos de interés, se han coloreado los keypoints sobre una copia del objeto original para visualizarlo.

3.4. Obtención de los descriptores

Tras obtener los puntos característicos de la escena y el objeto se generan los descriptores para cada usando el algoritmo FPFH implementado por open3D [12]. Este requiere que se hayan calculado las normales de la nube previamente y los parámetros del radio y vecindad sobre el que se desea calcular el descriptor. Es importante que los descriptores de la escena y el objeto sean del mismo tamaño para poder realizar el proceso de emparejamiento mediante RANSAC. Sobre el descriptor de cada nube se itera usando la búsqueda mediante KD-TreeFlann [13] según los puntos característicos con el fin de simplificar el descriptor, obteniendo un descriptor exclusivamente de los puntos clave de la nube.

3.5. Cálculo de los emparejamientos y transformación inicial

Para calcular la transformación que alinee la nube de puntos del objeto dentro de la escena se necesita en primer lugar obtener todas las correspondencias entre el descriptor de keypoints del objeto y el descriptor de keypoints de la escena. Empleando el método *search_knn_vector_xd* [14] se obtienen las correspondencias iniciales. Para filtrar las buenas correspondencias se emplea el método de RANSAC basado en registro de correspondencias [15], donde con el ajuste de los umbrales (*edge_length, distance, normal_angle*), se obtiene la transformación rígida acorde a los *inliers* (correspondencias buenas).

3.6. Refinamiento de la transformación

La matriz de transformación obtenida mediante RANSAC es un subóptimo, es decir, puede ser la solución óptima a la alineación de la nube del objeto sobre la escena, pero no es seguro de que siempre se cumpla. Esto se debe al componente de muestreo aleatorio intrínseco al algoritmo de RANSAC, falsos positivos en las correspondencias y el hecho de que RANSAC se basa exclusivamente en maximizar el número de *inliers* del modelo. Para refinar la solución obtenida se emplea el algoritmo ICP implementado en la librería open3D [16]. Este algoritmo minimiza el error de alineación de forma iterativa entre ambas nubes para obtener una matriz más precisa. Para emplear ICP es necesario un subóptimo de partida, un umbral de distancia, un criterio de convergencia (límite de iteraciones) y método de estimación (en este caso transformación punto-plano).

4. Experimentación

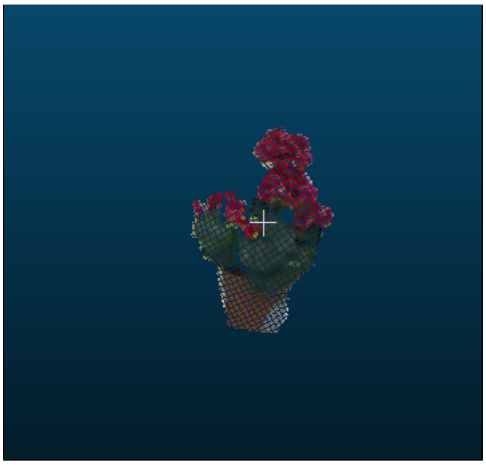


Figura 1. *Planta a detectar*

Para lograr una detección y alineación precisa de los objetos, se ha llevado a cabo un proceso iterativo de experimentación, con el fin de ajustar los parámetros clave en cada etapa del *pipeline* y optimizar los resultados obtenidos. En esta sección se muestra la alineación de la nube de la planta (figura 1) en una escena mediante el proceso descrito en 3. En el anexo se incluyen el resto de imágenes resultado de aplicar este proceso, una solución para la segmentación 3D de un objeto junto a reconocimiento del mismo en otra escena y el enlace a todo el código implementado.

Eliminación de planos

Para la eliminación de planos, se utiliza un método basado en RANSAC, el cual permite segmentar superficies planas en la nube de puntos mediante un parámetro de distancia umbral. Este valor determina la tolerancia máxima para considerar que un punto pertenece al plano estimado.



Figura 2. *Comparación de la escena tras eliminar planos dominantes*

Al aumentar este umbral, se incrementa el grosor aceptado del plano, lo que resulta útil para absorber pequeñas irregularidades causadas por ruido del sensor. Sin embargo, en zonas críticas como la parte inferior (la mesa), este parámetro se reduce para evitar eliminar puntos relevantes pertenecientes a los objetos de interés. En la figura 2 se observa el resultado de eliminar los 3 planos dominantes en la escena original (izq.) y la nube de puntos resultante (derecha).

Generación de keypoints

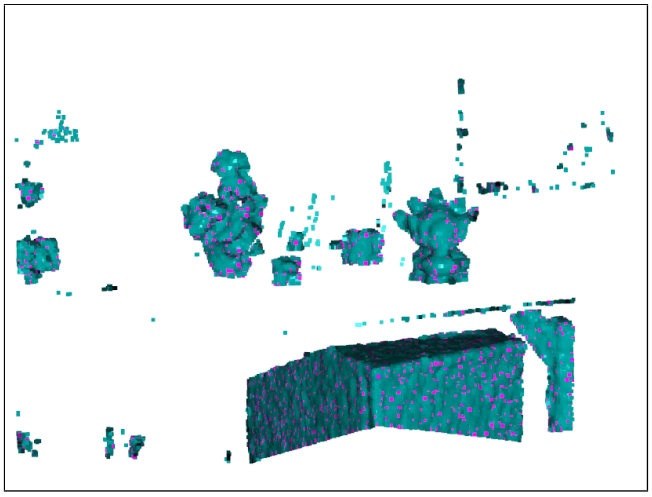


Figura 3. *Keypoints de la escena*

En la generación de keypoints para la nube principal (figura 3), se han ajustado los parámetros de muestreo para obtener una distribución equilibrada. Se busca que los puntos clave estén suficientemente espaciados, no sean redundantes y estén situados en regiones representativas de la geometría.

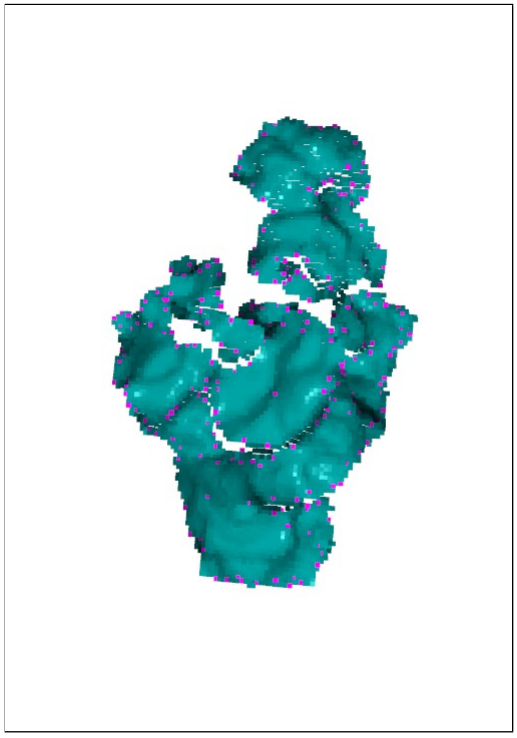


Figura 4. *Keypoints de la planta*

Para los objetos individuales, se sigue un criterio similar, aunque se requiere adaptar los parámetros a la geometría específica de cada objeto. Estas pequeñas variaciones son determinantes para lograr una detección más precisa y robusta (parámetros descritos anteriormente en 3.3). En la figura 4 se observan los puntos característicos detectados en la planta.

Emparejamiento y verificación de correspondencias

En el proceso de emparejamiento entre keypoints de la escena y los objetos, se utiliza un método basado en correspondencias geométricas. Este método implementa varios verificadores que permiten filtrar correspondencias no válidas y mejorar la calidad de la transformación estimada.

Los verificadores utilizados son:

- **Longitud de borde:** Comprueba que las proporciones relativas entre las distancias de pares de puntos se mantengan tras la transformación, conservando así la coherencia estructural.
- **Distancia:** Evalúa que la distancia euclidiana entre puntos correspondientes, después de aplicar la transformación, esté por debajo de un umbral predefinido.
- **Vectores normales:** Compara las normales asociadas a los puntos emparejados y verifica que el ángulo entre ellas no supere un umbral definido, garantizando la coherencia geométrica local.



Figura 5. *Resultado de la transformación obtenida con RANSAC.*

Estas comprobaciones aumentan la robustez del registro y permiten descartar falsos positivos manteniendo las correspondencias correctas como *inliers* del modelo. En la figura 5 se observa la transformación inicial obtenida para la planta. Se demuestra con este ejemplo que el resultado proporcionado por RANSAC es un subóptimo de la transformación deseada.

Refinamiento con ICP



Figura 6. *Resultado de aplicar ICP para la planta.*

La transformación inicial estimada mediante RANSAC es una buena aproximación de inicial para el algoritmo *Iterative Closest Point* (ICP). El parámetro más crítico a la hora de obtener una transformación óptima mediante ICP es el umbral de distancia, debido a que las nubes de los objetos son relativamente pequeñas y no se deben considerar puntos lejanos del objeto. En la figura 6 se observa el efecto de aplicar ICP para alinear con mayor precisión la planta dentro de la escena.

5. Conclusiones

La práctica desarrollada ha permitido implementar y analizar en profundidad un pipeline clásico de percepción 3D basado en nubes de puntos, utilizando herramientas como Open3D. A lo largo de las distintas etapas —desde el preprocesamiento de la nube hasta el refinamiento final con ICP— se ha demostrado la eficacia de estos métodos tradicionales para la alineación precisa de objetos tridimensionales en escenas complejas.

La experiencia práctica ha permitido observar cómo la elección adecuada de parámetros y el ajuste en cada etapa del proceso influyen directamente en la calidad del registro. Además, se ha dejado clara la importancia del preprocesamiento (como la eliminación de planos dominantes y la reducción de densidad) para optimizar el rendimiento y mejorar la estabilidad del emparejamiento. La detección de objetos 3D pequeños y aproximables a geometrías primitivas es la tarea más complicada debido a que se obtienen pocos keypoints sobre su volumen lo cual dificulta encontrar una buena transformación sobre la escena. Esto se ha verificado con objetos como la taza (incluida en el anexo).

Pese a las limitaciones del enfoque tradicional, como la sensibilidad al ruido, falta de capacidad para generalizar o la dependencia de buenos puntos clave; este tipo de técnicas continúan siendo relevantes y útiles como base para múltiples aplicaciones en robótica, visión artificial y reconstrucción 3D.

Referencias

[1] Vandal. (2022, octubre 10). Ubisoft lleva años recreando a la perfección monumentos en Assassin's Creed y estos son algunos excelentes ejemplos. Vandal.

[2] Zhong, Y. (2009). Intrinsic Shape Signatures: A Shape Descriptor for 3D Sensor Data. Proceedings of the International Conference on Computer Vision (ICCV), 689–696.

[3] Rusu, R. B., Blodow, N., & Beetz, M. (2009). Fast Point Feature Histograms (FPFH) for 3D Registration. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 3212–3217.

[4] Fischler, M. A., & Bolles, R. C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Communications of the ACM, 24(6), 381–395.

[5] Besl, P. J., & McKay, N. D. (1992). A Method for Registration of 3-D Shapes. IEEE Transactions on Pattern Analysis and Machine Intelligence, 14(2), 239–256.

[6] Wang & Solomon, "Deep Closest Point: Learning Representations for Point Cloud Registration", ICCV 2019.

[7] Zeng et al., "3DMatch: Learning Local Geometric Descriptors from RGB-D Reconstructions", CVPR 2017.

[8] Zhang & Singh, "LOAM: Lidar Odometry and Mapping in Real-time", RSS 2014.

[9] Huang et al., "PRIN: Pointwise Rotation-Invariant Network for Robust Point Cloud Registration", CVPR 2021

[10] Open3D. (n.d.). ISS keypoint detector. Open3D Documentation. Disponible en: `https://www.open3d.org/docs/release/tutorial/geometry/iss_keypoint_detector.html`

[11] Open3D. (n.d.). estimate_normals. Open3D Documentation. Disponible en: `https://www.open3d.org/docs/0.7.0/python_api/open3d.geometry.estimate_normals.html`

[12] Open3D. (n.d.). compute_fpfh_feature — Open3D Documentation. Disponible en: `https://www.open3d.org/docs/release/python_api/open3d.pipelines.registration.compute_fpfh_feature.html`

[13] Open3D. (n.d.). KDTreeFlann — Open3D Documentation. Disponible en:`https://www.open3d.org/docs/release/python_api/open3d.geometry.KDTreeFlann.html`

[14] Open3D. (n.d.). search_knn_vector_xd — Open3D Documentation. Disponible en: https://www.open3d.org/docs/release/python_api/open3d.geometry.KDTreeFlann.html#open3d.geometry.KDTreeFlann.search_knn_vector_xd

[15] Open3D. (n.d.). registration_ransac_based_on_correspondence — Open3D Documentation. Disponible en: https://www.open3d.org/docs/release/python_api/open3d.pipelines.registration.registration_ransac_based_on_correspondence.html

[16] Open3D. (n.d.). registration_icp — Open3D Documentation. Disponible en: https://www.open3d.org/docs/release/python_api/open3d.pipelines.registration.registration_icp.html

6. Anexo

6.1. Código fuente

Práctica 2. Detección de objetos 3D mediante el pipeline tradicional - Aitor R.B. et Juan P.F.

6.2. Experimentación realizada con otros objetos

A continuación se muestran los resultados paso a paso obtenidos para otros dos objetos diferentes en la misma escena empleada en 4

6.2.1. *Piggy*



Figura 6.2.1.1. *Objeto Piggy*

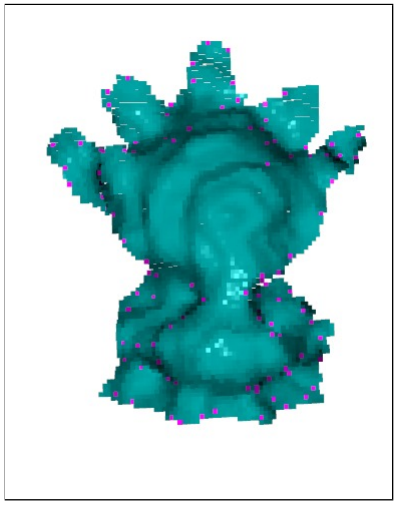


Figura 6.2.1.2. *Keypoints detectados para Piggy*



Figura 6.2.1.3. *Transformación inicial obtenida con RANSAC para Piggy*

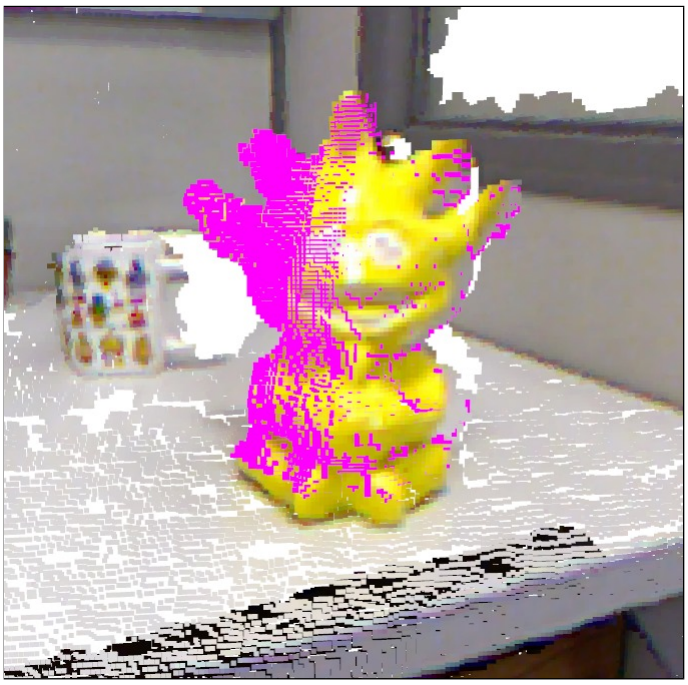


Figura 6.2.1.4. *Transformación final mediante ICP para Piggy*

6.2.2. *Mug*

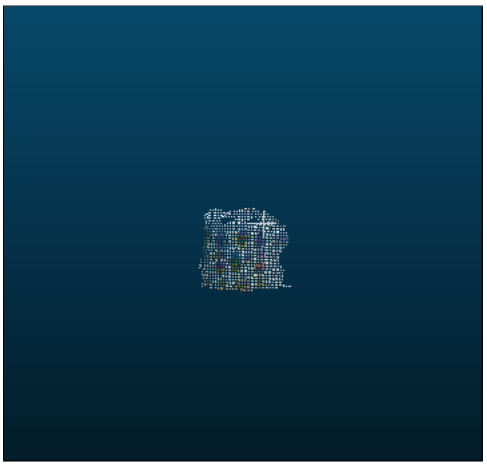


Figura 6.2.2.1. *Objeto Mug*



Figura 6.2.2.2. *Keypoints detectados para Mug*

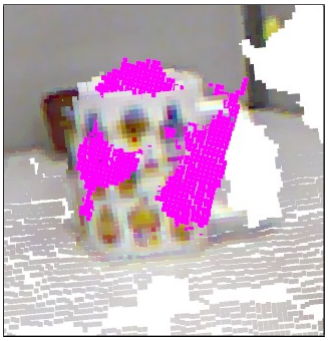


Figura 6.2.2.3. *Transformación inicial obtenida con RANSAC para Mug*

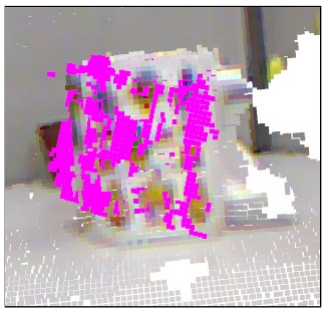


Figura 6.2.2.4. *Transformación final con ICP para Mug*

6.3. Segmentación 3D de un objeto - Charmander

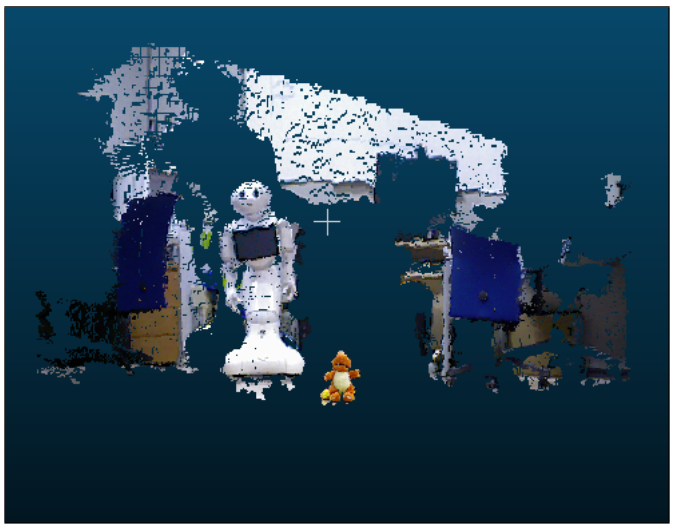


Figura 6.3.1. *Escena de Charmander*

Mediante la función `segmentObj` lleva a cabo el proceso completo de segmentación de un objeto 3D a partir de la nube de puntos de entrada (figura 6.3.1). El objetivo es segmentar el objeto en esa escena y reconocerlo en otra diferente. Sus pasos principales son:

1. **Reducción de la densidad:** aplica un muestreo por vóxel (`downsample_pcd`) para disminuir el número de puntos y agilizar el cálculo.

2. **Eliminación de elementos de fondo:** elimina iterativamente los planos principales de la nube usando RANSAC, quedándose solo con los puntos que no pertenecen a dichos planos.

3. **Segmentación por crecimiento de regiones:** a partir de la nube depurada, agrupa puntos en clústeres según la afinidad de sus normales y su cercanía espacial. Se emplea un umbral de puntos mínimo para considerar un cluster y así simplificar la escena.

4. **Filtrado de primitivas:** descarta aquellos clústeres que correspondan a formas geométricas simples (planos o líneas), conservando solo las agrupaciones que representan el objeto deseado. Se emplean para ello tanto la segmentación de planos basada en RANSAC como PCA.

5. **Salida:** guarda la nube de puntos resultante y la devuelve como la representación segmentada del objeto.

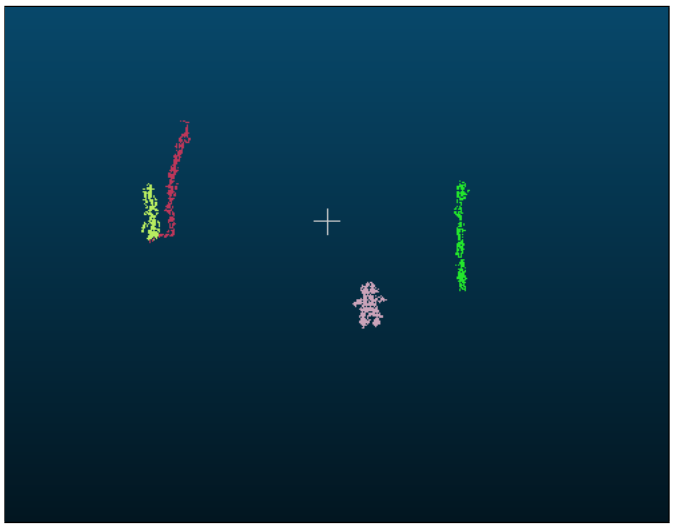


Figura 6.3.2. *Clusters detectados*

En la figura 6.3.2. se muestra el resultado tras aplicar la eliminación de planos principales y agrupar por *clusters* los puntos obtenidos.

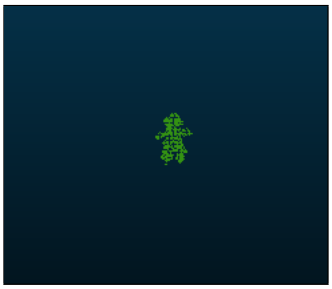


Figura 6.3.3. *Charmander segmentado*

Para eliminar el resto de clusters y conservar el del objeto se realiza un nuevo filtrado a través de RANSAC y posterior análisis con PCA obteniendo la nube de puntos de la figura 6.3.3. Sin embargo, debido al muestreo aleatorio que emplea RANSAC no siempre se obtiene un cluster final exlcusivamente con los puntos del charmander.

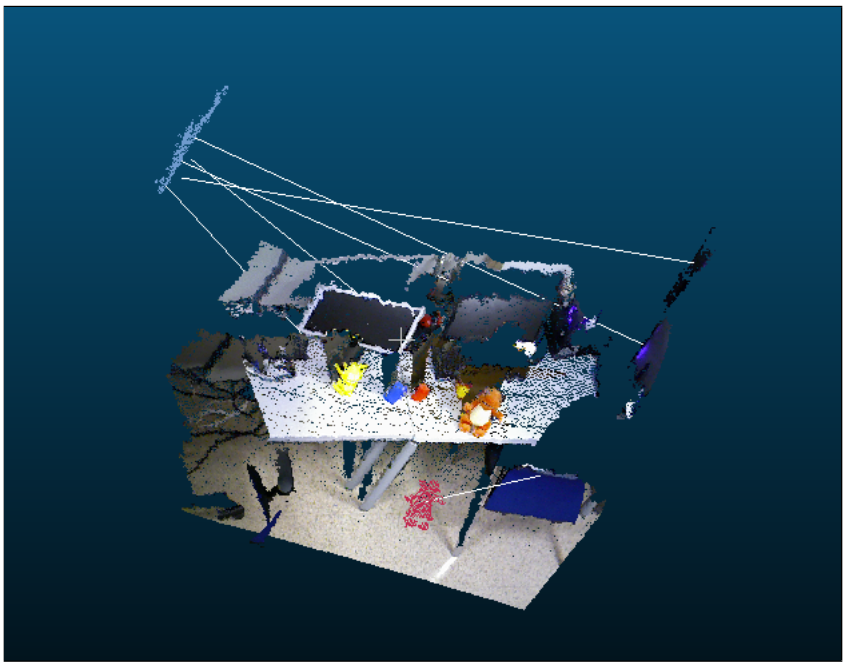


Figura 6.3.4. *Correspondencias obtenidas en uso del pipeline tradicional*

En la figura 6.3.4. se observa que la nube segmentada también incluye un cluster que no corresponde al charmander (cluster azul) y además, el número de correspondencias (líneas blancas) entre el descriptor de keypoints de la escena y el descriptor de keypoints del objeto es muy pequeño y de baja calidad.

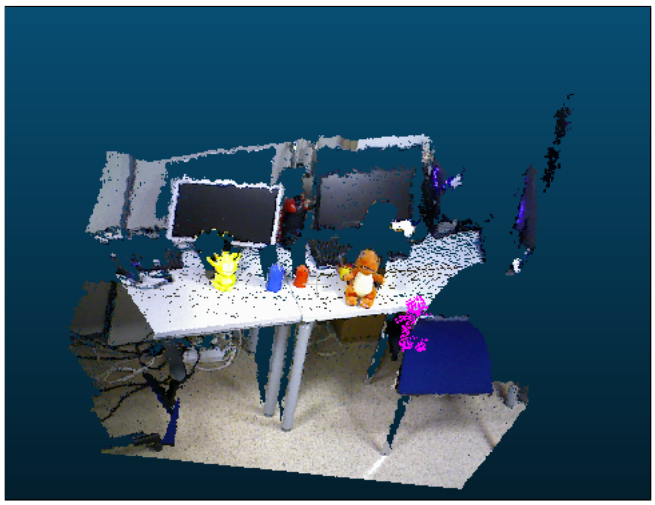


Figura 6.3.5. *Transformación para una escena diferente a la figura 6.3.1*

Consecuentemente, a pesar de haber segmentado bien en una escena el charmander, no se puede encontrar una transformación próxima a la óptima mediante el pipeline desarrollado para alinear su nube en otra escena diferente (figura 6.3.5). Sin embargo, en la escena original de la figura 6.3.1 sí se consigue reconocer el objeto y encontrar una transformación óptima con el pipeline desarrollado como se observa en la figura siguiente, donde se han coloreado en rosa los puntos de la nube segmentada y transformada sobre la escena original.

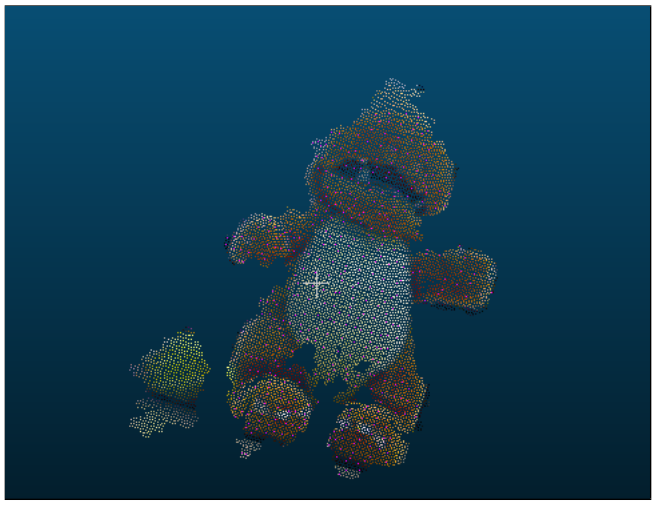


Figura 6.3.6. *Detalle de la escena original con charmander segmentado*